

Lab 4: VibeCoding - Chat History

Assigned repo: #56 SETAPESU26/56_tug_of_war (Pygame tug-of-war). LLM used: Claude (claude.ai).

Format note: user messages are reproduced verbatim. Claude's replies are condensed to the diagnosis, the key commands and the code that was used. Screenshots and pasted files are listed in brackets.

USER

My number is 56. help me clone this repo into an existing repo (SE LAB) as lab 4 and start working on it as specified in the instruction manual

[attached: Lab_4_VibeCoding_Student_handout.pdf, VibeCodingGitRepoLinks.pdf]

CLAUDE (condensed)

Number 56 maps to https://github.com/SETAPESU26/56_tug_of_war. The README describes one deliberate bug plus three features: Task 1 fix the input-lock deadlock, Task 2 computer panic surge, Task 3 rope tension and puller lean, Task 4 match timer and Sudden Death. A plain git clone inside SE-Lab would create a nested .git, so the repo is cloned to a temp folder, its .git removed, and the files copied into Lab-4/.

Commands

```
git clone https://github.com/SETAPESU26/56_tug_of_war.git ../tug_temp
rm -rf ../tug_temp/.git
mkdir -p Lab-4/tug_of_war Lab-4/videos Lab-4/chat-history
cp -r ../tug_temp/* Lab-4/tug_of_war/
rm -rf ../tug_temp
```

USER

i can't get the bug... as in i'm pressing a and d together, but the rope isn't freezing

CLAUDE (condensed)

Pressing both at the same instant is not the trigger. The README describes a rolling overlap of alternating keys. Claude suggested a temporary diagnostic print in `handle_event` to see the key events.

USER

[pasted the original game/game_engine.py]

[pasted: full original game_engine.py]

CLAUDE (condensed)

Diagnosis: the code never freezes permanently. `last_key` is only updated in the same branch that sets `is_pull_locked`, so the unlock always fires. The real bug is dropped keystrokes: while one key is held, the lock ignores the other key's KEYDOWN, so overlapping alternating presses are swallowed. `event.key != last_key` already enforces alternation, so the lock is redundant and should be removed.

Diagnostic used to reproduce (temporary)

```
if event.key in (pygame.K_a, pygame.K_d) and self.is_pull_locked:
    print(f"DROPPED {pygame.key.name(event.key)} (locked by {pygame.key.name(self.last_key)})")
```

Task 1 fix (game_engine.py, handle_event)

```
if event.type == pygame.KEYDOWN:
    if event.key in (pygame.K_a, pygame.K_d) and event.key != self.last_key:
        self.ropes.pull_left(1.0)
        self.last_key = event.key
```

USER

i need to commit the changes after each task...

CLAUDE (condensed)

Make a baseline commit of the untouched game first, add a .gitignore for __pycache__, then commit after each task with one message per task.

USER

how do i revert a commit? i used the baseline commit after the edit for task 1

CLAUDE (condensed)

git reset --mixed HEAD~1 undoes the commit but keeps the files. Restore the original game_engine.py from the assigned repo, make the baseline commit, then restore the fixed file and commit it as Task 1.

USER

start from scratch - delete the repo, push with a commit message and i'll start again

CLAUDE (condensed)

Assumed 'repo' meant only the Lab-4 folder (not SE-Lab, which holds earlier labs). Removed Lab-4 with git rm, committed and pushed, then gave a clean restart sequence.

USER

there's nothing in the tug of war folder

CLAUDE (condensed)

The earlier command block was not chained, so a failed clone left an empty folder. Re-did it with && chaining and /tmp as the temp location so the failing step is visible.

USER

alright this is done.... now take it from the top - everything step by step

CLAUDE (condensed)

Laid out six steps: (1) baseline check, bug repro and before video, (2) Task 1, (3) Task 2, (4) Task 3, (5) Task 4, (6) after video, chat export and final push.

USER

all good

[screenshot: terminal showing repeated 'DROPPED d (locked by a)' lines, game_engine.py open in IDLE]

CLAUDE (condensed)

Bug reproduced. Step 2: replace the lock block with the alternation-only check, delete is_pull_locked from __init__ and reset(), verify with grep, test, commit.

USER

done ig

[screenshot: terminal showing Task 1 commit 455f977 pushed; Before.mp4 in Finder]

CLAUDE (condensed)

Task 1 pushed. Cleaned up tracked .DS_Store files with a root .gitignore and git rm --cached. Asked for rope.py to build Task 2.

USER

here you go

[pasted: game/rope.py]

CLAUDE (condensed)

Task 2 design: compute progress = how far the flag is from center toward the player's goal (0 to 1). Past a threshold the computer's pull cooldown drops from 180 ms to a shorter value. Cooldown was chosen over strength because

pull_right truncates with int().

Task 2 (game_engine.py)

```
# in __init__
self.surge_threshold = 0.35
self.surge_cooldown = 100      # later tuned by the student to 130
self.computer_surgling = False

# in update()
center_x = self.rope.screen_width // 2
progress = (center_x - self.rope.marker_x) / (center_x - self.rope.left_win_x)
surgling = progress >= self.surge_threshold
self.computer_surgling = surgling
cooldown = self.surge_cooldown if surgling else self.computer_pull_cooldown
if now - self.last_computer_pull >= cooldown:
    computer_variance = random.uniform(0.7, 1.2)
    self.rope.pull_right(computer_variance)
    self.last_computer_pull = now
```

USER

i updated the surge to 130... let's move on... everything works now

CLAUDE (condensed)

Task 2 committed. Task 3 Part A: add import math and a tension() method to Rope (0 at center, 1 at a goal line), and draw the rope as a polyline whose points are offset by a sine wave scaled by tension and time. Part B (puller lean) needs player.py.

USER

this does not look right...

[screenshot: SyntaxError in rope.py line 40, render() edited inside the old pygame.draw.line(call]

CLAUDE (condensed)

Cause: the replacement was inserted inside the opening pygame.draw.line(call instead of replacing it, because the instruction said to replace the call. Fix: replace the entire render() method.

USER

yes the rope is wobbling as the red bar moves towards either side

CLAUDE (condensed)

Part A works. Asked for player.py for the lean.

USER

here you go

[pasted: game/player.py]

CLAUDE (condensed)

Lean design: one value from -1 to 1 from the flag position; each puller rotates its body and head around its feet. The player leans away from the rope, the computer leans with it, flipping when the flag crosses center. GameEngine.render() passes the lean value into each Puller.render().

USER

tha players aren't leaning

CLAUDE (condensed)

Three checks: (1) edits saved and present via grep, (2) a temporary print of the lean value, (3) the tilt may simply be too subtle at 25 degrees.

USER

error showing up

[screenshot: *AttributeError 'Rope' object has no attribute 'max_lean_deg' at rope.py line 40*]

CLAUDE (condensed)

The Puller's render() code (with self.max_lean_deg) had been pasted into rope.py. py_compile printed OK because the code was valid, just in the wrong file. Fix: restore the rope's own render() and keep the lean code only in player.py.

USER

ok more errors are showing up.... just give me the 3 code cells (full code) and i'll copy paste it into the 3 files in the repo

CLAUDE (condensed)

Provided complete rope.py, player.py and game_engine.py with Tasks 1 to 3 combined and no diagnostics. rope.py and player.py are reproduced below. game_engine.py had the same content as the final version at the end of this document, minus the timer.

game/rope.py (final)

```

import math
import pygame

class Rope:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.center_y = screen_height // 2
        self.marker_x = screen_width // 2

        self.left_win_x = 180
        self.right_win_x = screen_width - 180
        self.pull_step = 12

    def pull_left(self, strength=1.0):
        self.marker_x -= int(self.pull_step * strength)

    def pull_right(self, strength=1.0):
        self.marker_x += int(self.pull_step * strength)

    def check_winner(self):
        if self.marker_x <= self.left_win_x:
            return "PLAYER"
        if self.marker_x >= self.right_win_x:
            return "COMPUTER"
        return None

    def tension(self):
        center_x = self.screen_width // 2
        half_span = center_x - self.left_win_x
        return min(1.0, abs(self.marker_x - center_x) / half_span)

    def reset(self):
        self.marker_x = float(self.screen_width // 2)
        self.velocity = 0.0

    def render(self, surface):
        t = pygame.time.get_ticks() / 1000.0
        amplitude = 8 * self.tension()
        points = [
            (x, self.center_y + amplitude * math.sin(x * 0.05 + t * 25))
            for x in range(60, self.screen_width - 60 + 1, 10)
        ]
        pygame.draw.lines(surface, (180, 140, 90), False, points, 10)

        pygame.draw.line(surface, (50, 200, 50),
            (self.left_win_x, self.center_y - 40),
            (self.left_win_x, self.center_y + 40), 4)
        pygame.draw.line(surface, (200, 50, 50),
            (self.right_win_x, self.center_y - 40),
            (self.right_win_x, self.center_y + 40), 4)
        pygame.draw.line(surface, (120, 120, 120),
            (self.screen_width // 2, self.center_y - 20),
            (self.screen_width // 2, self.center_y + 20), 2)

        flag_rect = pygame.Rect(int(self.marker_x) - 12, self.center_y - 24, 24, 48)
        pygame.draw.rect(surface, (230, 40, 40), flag_rect, border_radius=4)
        pygame.draw.rect(surface, (255, 255, 255), flag_rect, width=2, border_radius=4)

```

game/player.py (final)

```

import math
import pygame

class Puller:
    """Represents a puller character anchor on either side of the rope."""

    def __init__(self, x, y, color, label):
        self.x = x
        self.y = y
        self.color = color
        self.label = label
        self.font = pygame.font.SysFont(None, 24)
        self.max_lean_deg = 35

    def render(self, surface, lean=0.0):
        """Draw avatar tilted around its feet; lean in [-1, 1], positive = tilt right."""
        angle = math.radians(lean * self.max_lean_deg)
        pivot_x, pivot_y = self.x, self.y + 35 # feet

        def rot(px, py):
            dx, dy = px - pivot_x, py - pivot_y
            return (
                pivot_x + dx * math.cos(angle) - dy * math.sin(angle),
                pivot_y + dx * math.sin(angle) + dy * math.cos(angle),
            )

        corners = [
            rot(self.x - 20, self.y - 35),
            rot(self.x + 20, self.y - 35),
            rot(self.x + 20, self.y + 35),
            rot(self.x - 20, self.y + 35),
        ]
        pygame.draw.polygon(surface, self.color, corners)

        head_x, head_y = rot(self.x, self.y - 50)
        pygame.draw.circle(surface, (240, 210, 180), (int(head_x), int(head_y)), 16)

        label_surf = self.font.render(self.label, True, (240, 240, 240))
        surface.blit(label_surf, (self.x - label_surf.get_width() // 2, self.y + 45))

```

USER

okay yeah the players are leaning now... move on

CLAUDE (condensed)

Committed Task 3 (rope.py, player.py, game_engine.py). Task 4 design: record match_start, compute elapsed time in update(), set sudden_death after 45 s, and use one multiplier (1x normal, 2x Sudden Death) on both the keystroke pull and the computer's tick pull. The timer is drawn at the top, stops on game over and resets with R. For testing, the threshold was temporarily lowered to 5 s and restored to 45000 before committing.

game/game_engine.py (final, Tasks 1-4)

```

import random
import pygame
from game.rope import Rope
from game.player import Puller

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)")
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTER")

        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        self.computer_pull_cooldown = 180
        self.last_computer_pull = pygame.time.get_ticks()

        self.surge_threshold = 0.35 # fraction of the way from center to the player's goal
        self.surge_cooldown = 130 # ms between computer pulls while surging
        self.computer_surging = False

        self.sudden_death_ms = 45000 # match time before Sudden Death starts
        self.match_start = pygame.time.get_ticks()
        self.elapsed_ms = 0
        self.sudden_death = False

        self.font_big = pygame.font.SysFont(None, 48)
        self.font_small = pygame.font.SysFont(None, 26)
        self.font_timer = pygame.font.SysFont(None, 34)

    def pull_multiplier(self):
        return 2.0 if self.sudden_death else 1.0

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_a, pygame.K_d) and event.key != self.last_key:
                self.rope.pull_left(1.0 * self.pull_multiplier())
                self.last_key = event.key

    def update(self):
        if self.game_state != "PLAYING":
            return

        now = pygame.time.get_ticks()

        self.elapsed_ms = now - self.match_start

```

```

self.sudden_death = self.elapsed_ms >= self.sudden_death_ms

center_x = self.rope.screen_width // 2
progress = (center_x - self.rope.marker_x) / (center_x - self.rope.left_win_x)
surging = progress >= self.surge_threshold
self.computer_surging = surging

cooldown = self.surge_cooldown if surging else self.computer_pull_cooldown
if now - self.last_computer_pull >= cooldown:
    computer_variance = random.uniform(0.7, 1.2)
    self.rope.pull_right(computer_variance * self.pull_multiplier())
    self.last_computer_pull = now

result = self.rope.check_winner()
if result:
    self.winner = result
    self.game_state = "GAME_OVER"

def reset(self):
    self.rope.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
    self.computer_surging = False
    self.last_computer_pull = pygame.time.get_ticks()
    self.match_start = pygame.time.get_ticks()
    self.elapsed_ms = 0
    self.sudden_death = False

def render(self, screen):
    screen.fill((30, 32, 36))

    mud_rect = pygame.Rect(self.width // 2 - 120, self.height // 2 - 80, 240, 160)
    pygame.draw.rect(screen, (45, 38, 30), mud_rect, border_radius=12)

    self.rope.render(screen)

    center_x = self.width // 2
    momentum = (center_x - self.rope.marker_x) / (center_x - self.rope.left_win_x)
    lean = -max(-1.0, min(1.0, momentum))
    self.player.render(screen, lean)
    self.computer.render(screen, lean)

    # Match timer (top of screen)
    seconds = self.elapsed_ms / 1000.0
    if self.sudden_death:
        timer_text = f"{seconds:.1f}s - SUDDEN DEATH (x2 pulls)"
        timer_color = (255, 70, 70)
    else:
        timer_text = f"{seconds:.1f}s"
        timer_color = (240, 240, 240)
    timer_surf = self.font_timer.render(timer_text, True, timer_color)
    screen.blit(timer_surf, (self.width // 2 - timer_surf.get_width() // 2, 10))

    inst_surf = self.font_small.render(

```

```

        "Alternate [A] and [D] keys rapidly to pull!", True, (210, 210, 210)
    )
    screen.blit(inst_surf, (self.width // 2 - inst_surf.get_width() // 2, 40))

    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

        win_text = f"{self.winner} WINS!"
        color = (80, 220, 80) if self.winner == "PLAYER" else (240, 80, 80)
        text_surf = self.font_big.render(win_text, True, color)
        screen.blit(
            text_surf,
            (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 50)
        )

        restart_surf = self.font_small.render(
            "Press [R] to Play Again", True, (240, 240, 240)
        )
        screen.blit(
            restart_surf,
            (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10)
        )

```

USER

yes pls

CLAUDE (condensed)

Step 6: confirm Task 4 is committed and sudden_death_ms is back at 45000, record the after video (temporarily using 5 s for Sudden Death), export the chat as PDF, then make the final commit with the videos and chat-history folder and push. Do not raise a PR to SETAPESU26.

Final commit

```

git add Lab-4/videos Lab-4/chat-history
git commit -m "Lab-4: add before/after videos and chat history"
git push

```

USER

can you give me a pdf... i'm not able to print the chat history

CLAUDE (condensed)

Generated this PDF as a condensed transcript of the conversation.